

Estrategia unificada de observabilidad con OpenTelemetry

ADR y pipeline para un e-commerce multinube

Carlos Arturo Salazar Castaneda

Juan Sebastian Buitrago Romero

Juan Jose Velez Alvarez

Jhonatan Winston Sotelo

Santiago Eduardo Muñoz Castillo

22 de septiembre de 2026

Incluye archivos YAML, diagrama editable y guía de continuidad.

ADR-001. Adoptar OpenTelemetry para la telemetría

Estado: propuesto. Fecha: 22 de septiembre de 2026.

Contexto

La plataforma opera 12 microservicios en GKE y ECS, con 10.000 solicitudes/s. Necesita correlacionar trazas, métricas y logs sin depender de un proveedor. Lenguajes, distribución y spans por solicitud están por confirmar. Se sigue el formato de Nygard (2011).

Opciones consideradas

Opción	Evaluación para este escenario
OpenTelemetry	API y OTLP comunes; permite sustituir destinos. Exige operar Collectors y comprobar compatibilidad.
SDKs específicos del proveedor	Integración directa con funciones nativas; aumenta el trabajo de migración si el código depende de APIs exclusivas.
Agentes propietarios	Despliegue inicial sencillo cuando existe soporte; cobertura, configuración y costes dependen del producto.

Decisión

Adoptar APIs y SDKs OTel, con OTLP a gateways por nube. Instrumentar automáticamente las bibliotecas compatibles y manualmente el negocio. Enviar métricas y logs al backend local y trazas a Jaeger. Se consultó OTel 1.61.0; Collector y convenciones semánticas se versionan por separado (OpenTelemetry, s. f.-e).

Consecuencias

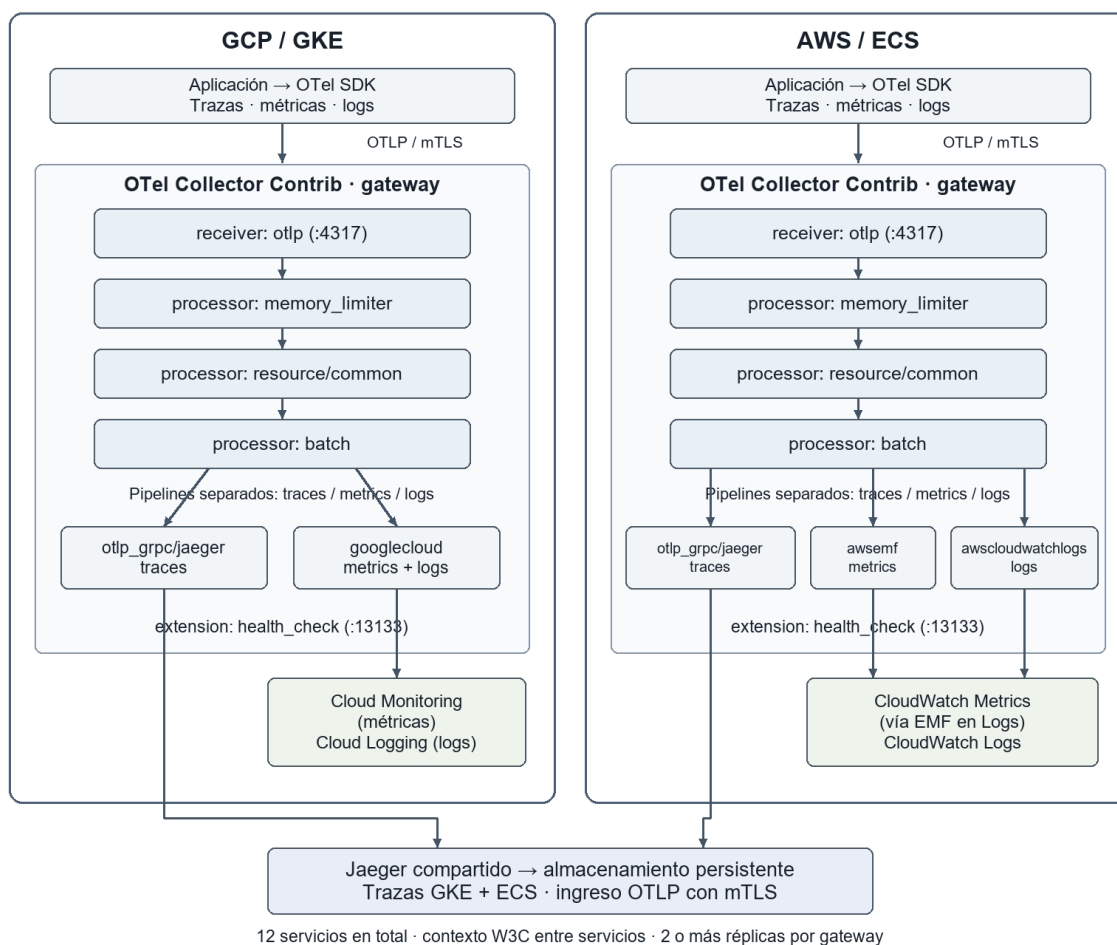
Se reduce el acoplamiento; consultas, alertas e históricos aún requieren migración. CPU, memoria, red y operación añaden costes que deben medirse. Se fija Collector Contrib 0.160.0; su exporter awscloudwatchlogs es alfa y requiere validación (OpenTelemetry, s. f.-b). Sampling y taxonomía quedan para los puntos 4 y 5.

Diseño del pipeline de telemetría

Cada aplicación envía al gateway de su nube: Deployment/Service en GKE y servicio con balanceador interno en ECS. Se parte de dos réplicas por gateway; la capacidad se validará bajo carga.

Figura 1

Pipeline multinube y componentes reales del Collector



Nota. Elaboración propia. Cada pipeline tiene su secuencia de processors; health_check no transporta datos. Jaeger recibe solo trazas (Jaeger, s. f.).

La unificación reside en la API, el protocolo y la propagación de contexto W3C, no en el backend. Cada señal tiene un destino por nube y las trazas de GKE y ECS convergen en el mismo Jaeger (OpenTelemetry, s. f.-a, s. f.-d).

Configuración del Collector

Base común a las dos nubes, combinada con un único perfil. Un componente solo se activa si aparece en service.pipelines (OpenTelemetry, s. f.-c).

```
receivers:      # rutas de certificado relativas a /etc/otel/tls
  otlp: {protocols: {grpc: {endpoint: 0.0.0.0:4317,
    tls: {cert_file: server.crt, key_file: server.key, client_ca_file: ca.crt}}}}
processors:
  memory_limiter: {check_interval: 1s, limit_mib: 768, spike_limit_mib: 192}
  resource/common: {attributes: [{key: service.namespace, value: ecommerce, action: insert}]}
  batch: {timeout: 1s, send_batch_size: 512, send_batch_max_size: 1024}
exporters:
  otlp_grpc/jaeger: {endpoint: "${env:JAEGER_OTLP_ENDPOINT}", sending_queue: {enabled: true,
    queue_size: 1000}, retry_on_failure: {enabled: true, max_elapsed_time: 300s},
    tls: {ca_file: jaeger-ca.crt, cert_file: client.crt, key_file: client.key}}
extensions: {health_check: {endpoint: 0.0.0.0:13133}}
service:
  extensions: [health_check]
  pipelines:
    traces: {receivers: [otlp], processors: [memory_limiter, resource/common, batch],
      exporters: [otlp_grpc/jaeger]}
```

Valores de ensayo para un contenedor de 1 GiB, no un dimensionamiento validado para 10k RPS. Los cinco archivos YAML se adjuntan como anexo digital.

Tabla 1

Destino de cada señal por nube

Señal	GCP / GKE (collector-gcp.yaml)	AWS / ECS (collector-aws.yaml)
Métricas	googlecloud, project \${env:GCP_PROJECT_ID} → Cloud Monitoring	awsemf, namespace Ecommerce, NoDimensionRollup, dimensiones [service.name, deployment.environment.name] → CloudWatch Metrics (EMF)
Logs	googlecloud, log.default_log_name ecommerce → Cloud Logging	awsccloudwatchlogs, grupo /ecommerce/application → CloudWatch Logs
Trazas	otlp_grpc/jaeger de la base	otlp_grpc/jaeger de la base; ambas nubes escriben en el mismo Jaeger

Nota. Elaboración propia. El punto 5 ampliará las dimensiones EMF de los SLIs.

Esta base reenvía todas las trazas: no define sampling ni aprueba retener el 100 % a 10k RPS, decisión del punto 4. Los SLIs se calcularán sobre métricas de las solicitudes elegibles: muestrear errores sesga las estadísticas de trazas (OpenTelemetry, s. f.-f).

Referencias

- Google Cloud. (s. f.). *Instrument for Cloud Trace*. Recuperado el 22 de septiembre de 2026, de <https://docs.cloud.google.com/trace/docs/setup>
- Jaeger. (s. f.). *APIs*. Recuperado el 22 de septiembre de 2026, de <https://www.jaegertracing.io/docs/2.21/architecture/apis/>
- Nygard, M. (2011, 15 de noviembre). *Documenting architecture decisions*. Cognitect. <https://cognitect.com/blog/2011/11/15/documenting-architecture-decisions>
- OpenTelemetry. (s. f.-a). *AWS CloudWatch EMF exporter* (versión 0.160.0) [Código fuente]. GitHub. <https://github.com/open-telemetry/opentelemetry-collector-contrib/tree/v0.160.0/exporter/awsemfexporter>
- OpenTelemetry. (s. f.-b). *AWS CloudWatch Logs exporter* (versión 0.160.0) [Código fuente]. GitHub. <https://github.com/open-telemetry/opentelemetry-collector-contrib/tree/v0.160.0/exporter/awscloudwatchlogsexporter>
- OpenTelemetry. (s. f.-c). *Configuration*. Recuperado el 22 de septiembre de 2026, de <https://opentelemetry.io/docs/collector/configuration/>
- OpenTelemetry. (s. f.-d). *Google Cloud exporter* (versión 0.160.0) [Código fuente]. GitHub. <https://github.com/open-telemetry/opentelemetry-collector-contrib/tree/v0.160.0/exporter/googlecloudexporter>
- OpenTelemetry. (s. f.-e). *OpenTelemetry specification 1.61.0*. Recuperado el 22 de septiembre de 2026, de <https://opentelemetry.io/docs/specs/otel/>
- OpenTelemetry. (s. f.-f). *Sampling*. Recuperado el 22 de septiembre de 2026, de <https://opentelemetry.io/docs/concepts/sampling/>